

第一章

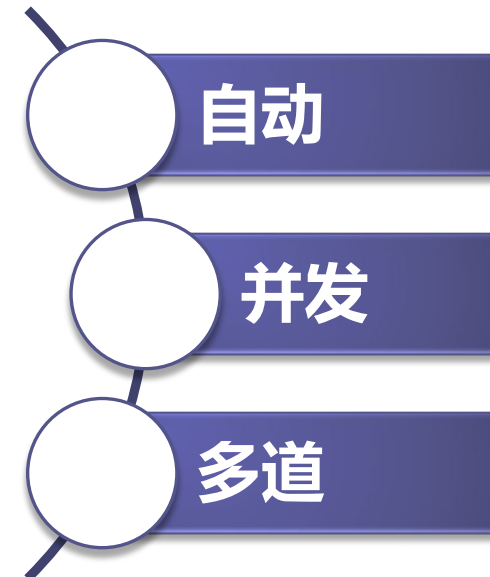
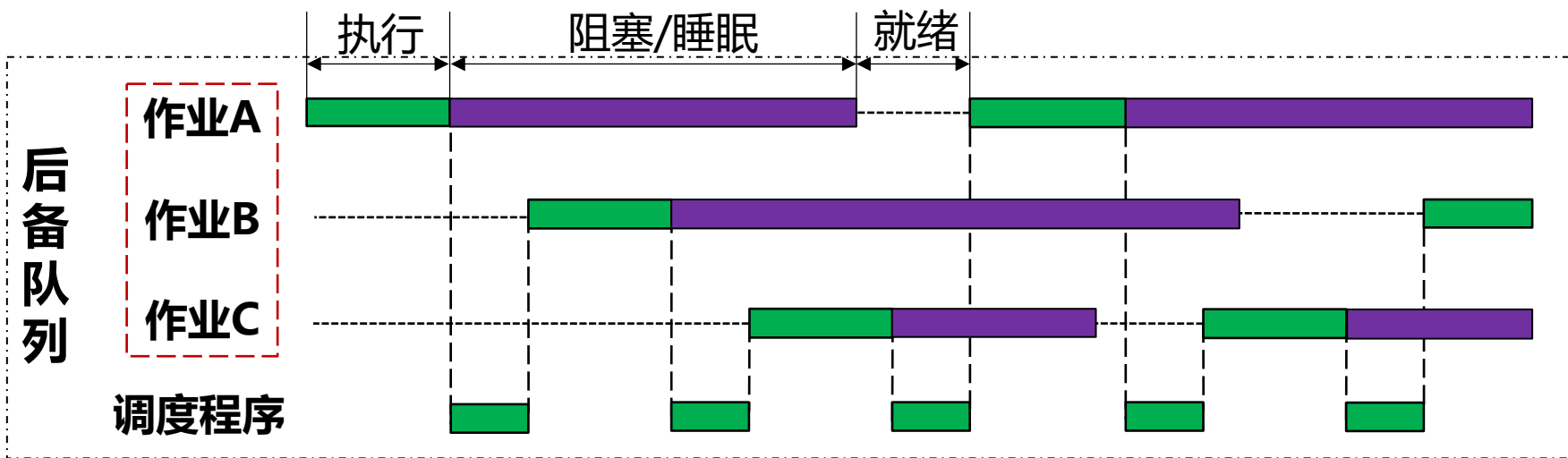
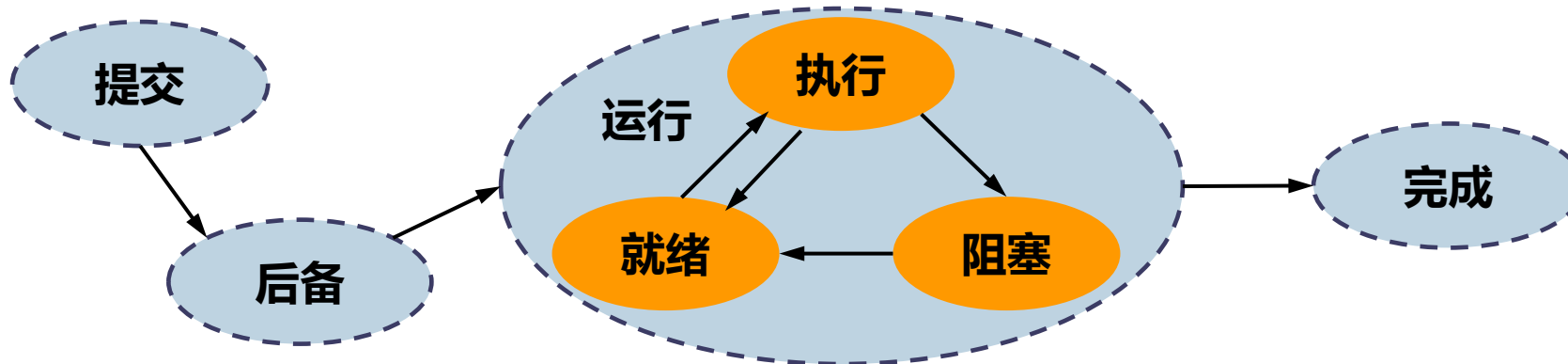
绪论



多道批处理系统 (60年代中期)



采用多道批处理技术后，作业从进入到退出系统大致经历四个阶段：



主要内容

- 1.1 设置操作系统的目的
- 1.2 操作系统的形成和发展
- 1.3 现代操作系统的功能与特征**
- 1.4 UNIX操作系统



现代操作系统的功能与特征



操作系统资源管理功能



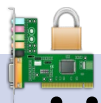
处理机管理

.....



存储器管理

.....



设备管理

.....



文件管理

.....



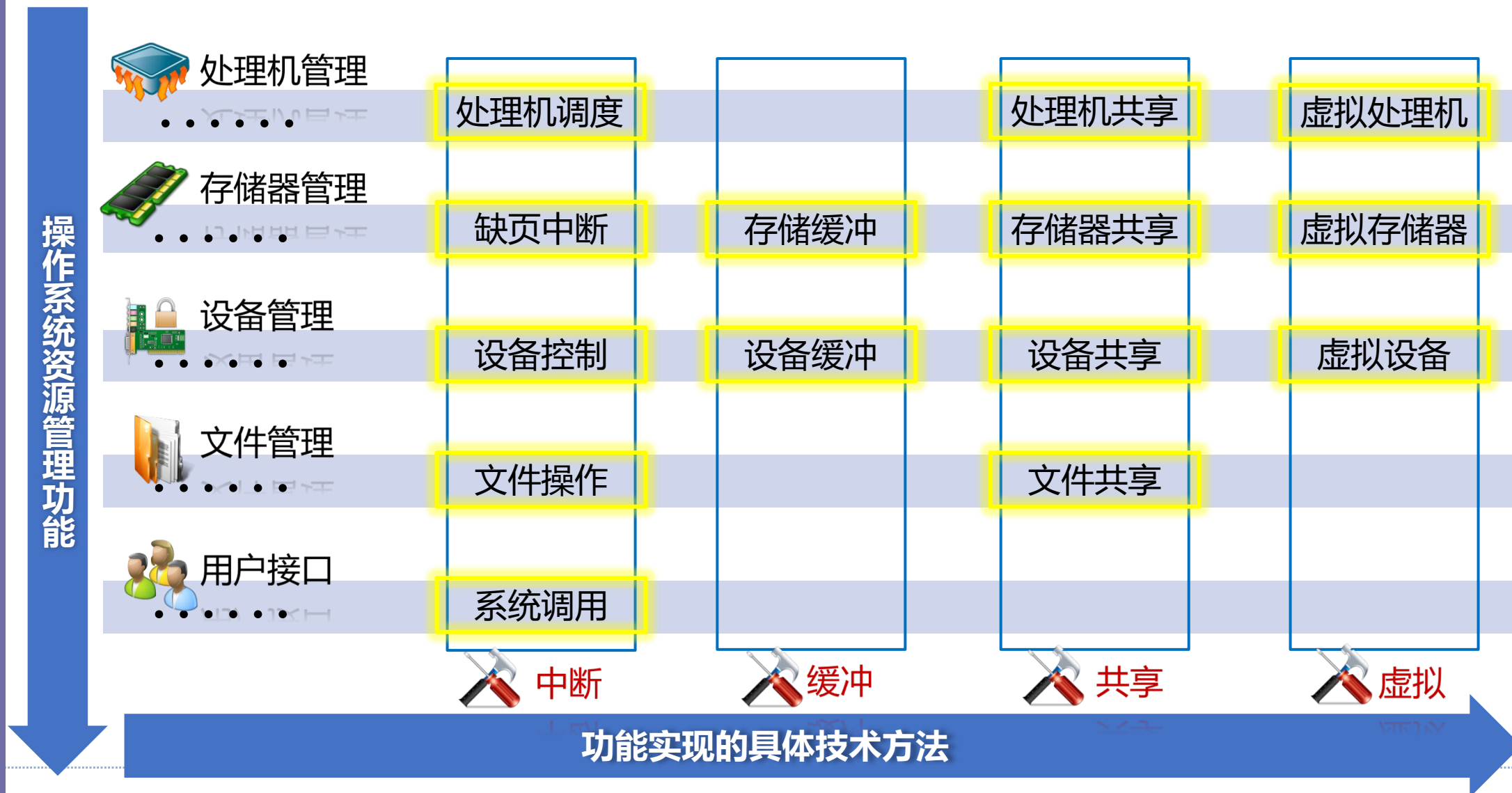
用户接口

.....

功能实现的具体技术方法

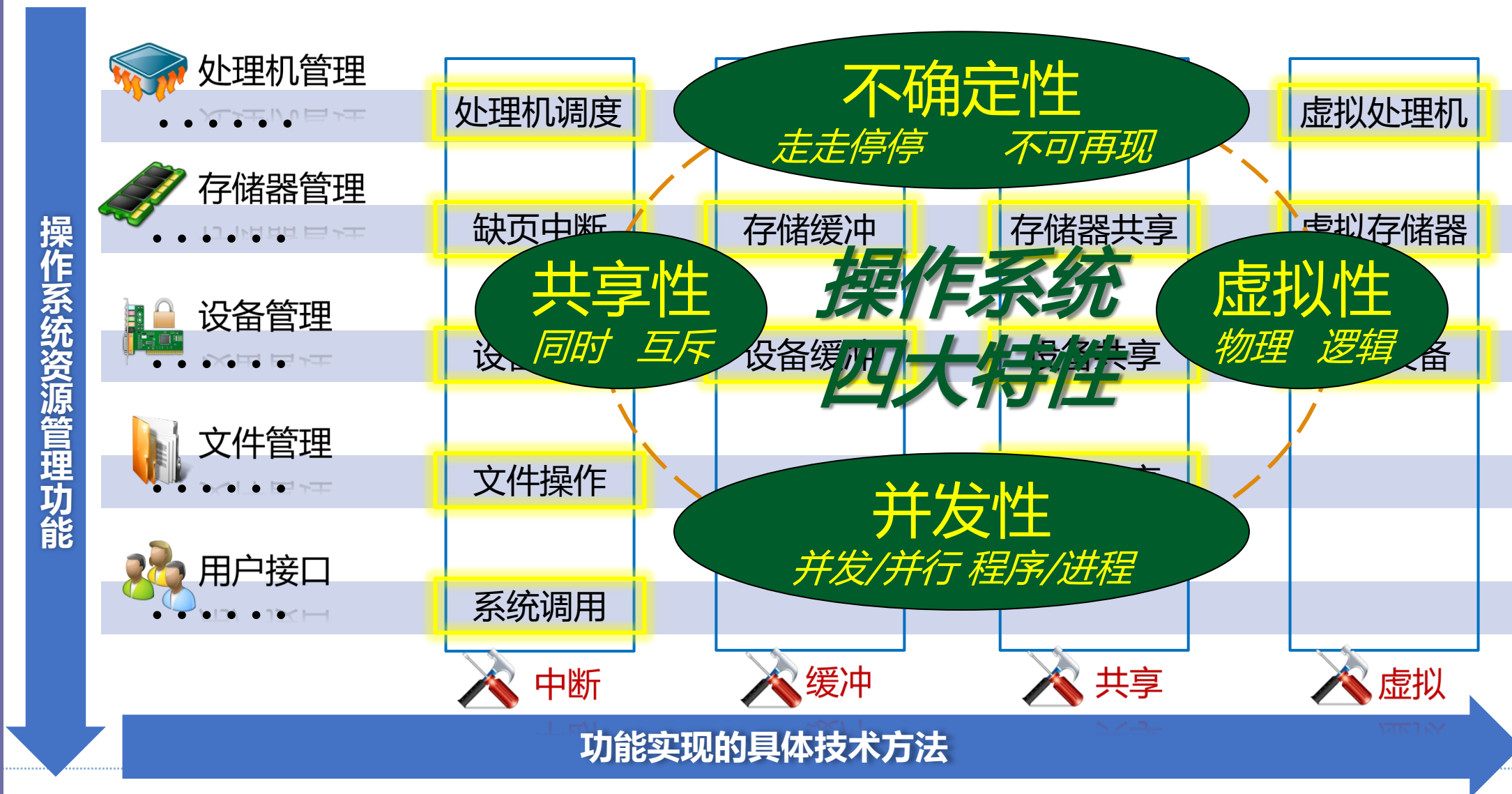


现代操作系统的功能与特征





现代操作系统的功能与特征



主要内容

- 1.1 设置操作系统的目的
- 1.2 操作系统的形成和发展
- 1.3 现代操作系统的功能与特征
- 1.4 UNIX操作系统**



UNIX的诞生



- 1 1965年, **Thompson**在Bell实验室参与开发一个称为Multics的新操作系统。Ken写了一个“star travel”游戏可执行于Multics之上。
- 2 Bell实验室退出Multics 项目后, 26岁的Thompson无事可做且玩游戏心切, 决定自己开发一个操作系统。在一台废弃的PDP-7机器上, 利用汇编语言, 汤普生只用一个月就编写完毕操作系统的内核。
- 3 1970年, 两人合作将UNIX移植到PDP-11上, 完成UNIX的第一个版本。



- 4 1973年, 为了程序移植的方便, **Ritchie** 开发出C语言。
- 5 1973年, 两人合作用C语言重写了UNIX。至此, 开启了UNIX和C的辉煌。
- 6 1983年, 两人被授予图灵奖。
- 7 2000年12月时, **Thompson**退休, 离开贝尔实验室, 成为了一名飞行员。



Ken Thompson



Dennis M. Ritchie



精巧的核心与丰富的实用层

内核：进程管理、存储管理、设备管理、文件管理等。内核设计精干简洁。只占用很小的内存并常驻内存。

核外程序：语言处理程序、编辑程序、调试程序、系统状态监控和文件管理程序、命令解释程序shell。

使用灵活的用户界面

命令程序设计语言Shell：是一种命令语言，也是一种程序设计语言。

程序接口：即：系统调用，包括汇编语言和C语言的。

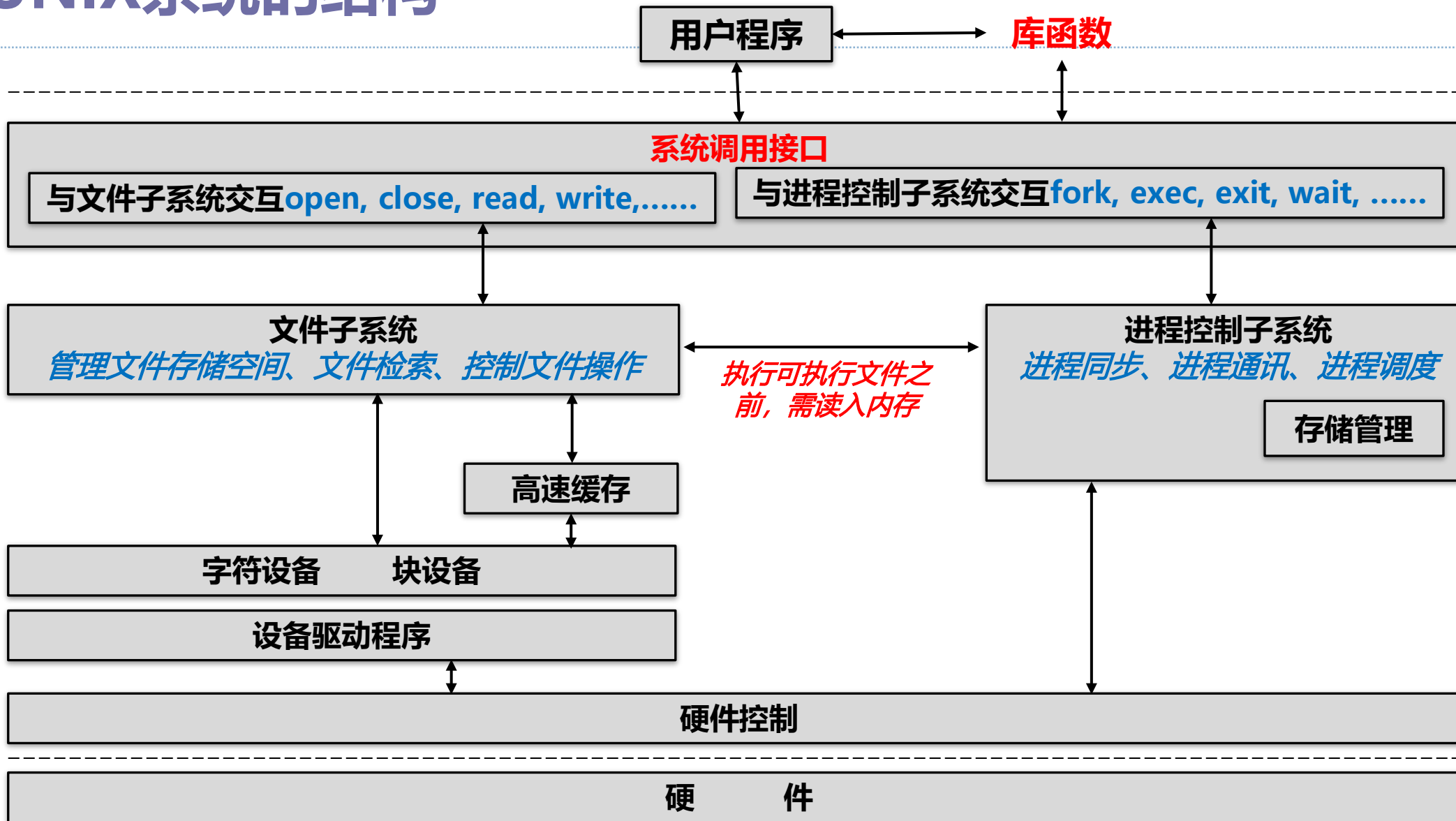
树形结构的文件系统

文件和设备统一看待

良好的移植性

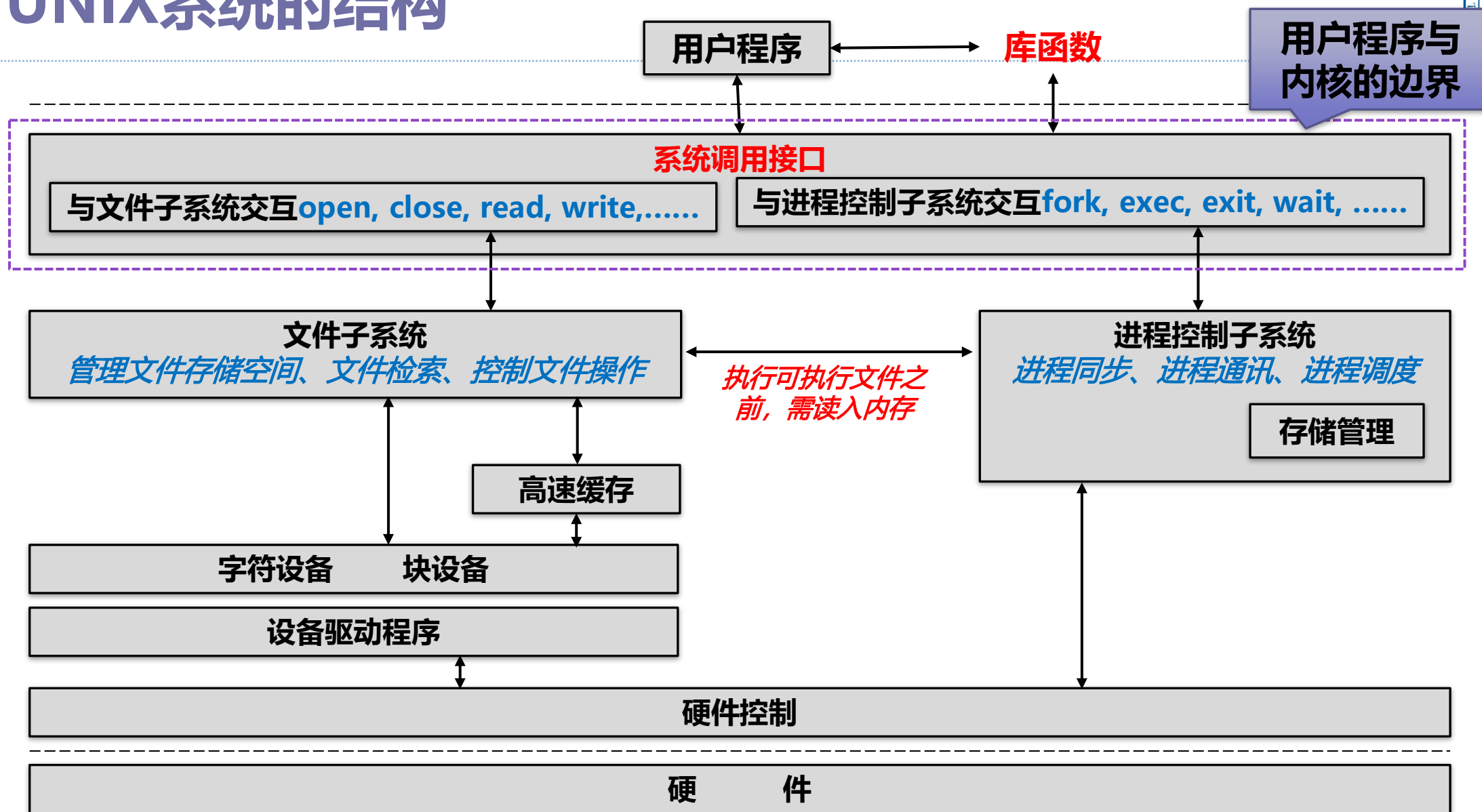


UNIX系统的结构



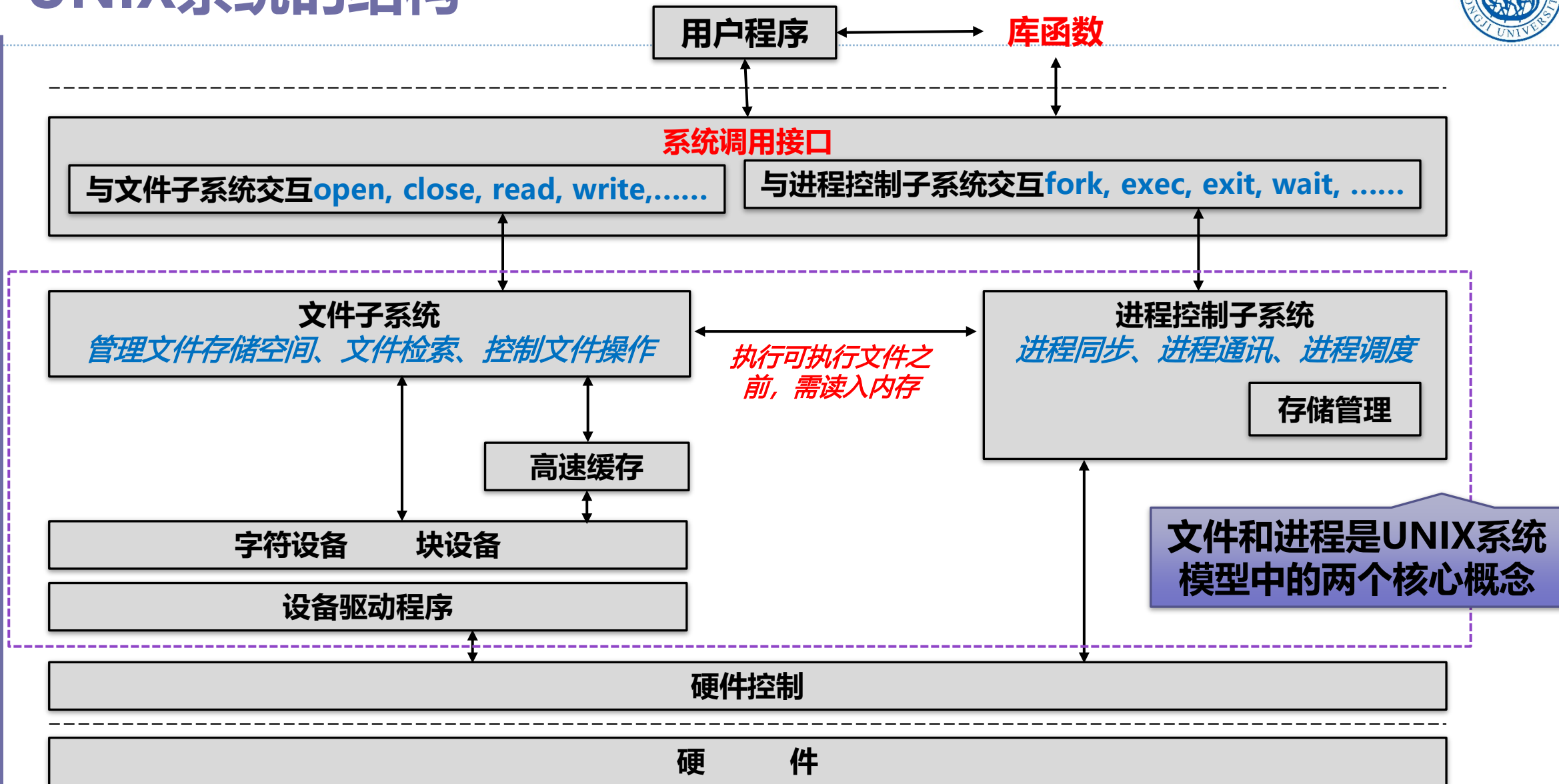


UNIX系统的结构



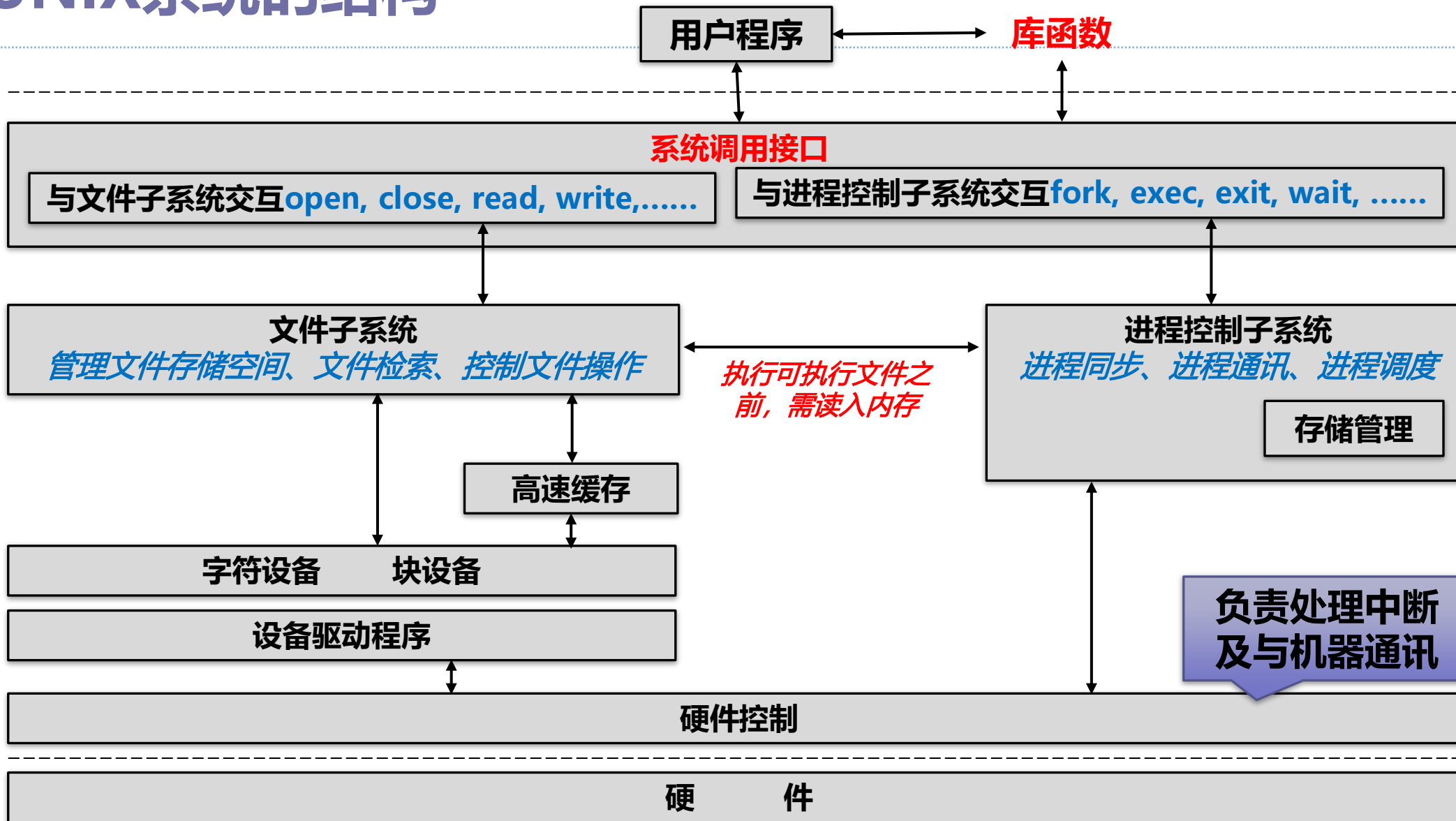


UNIX系统的结构



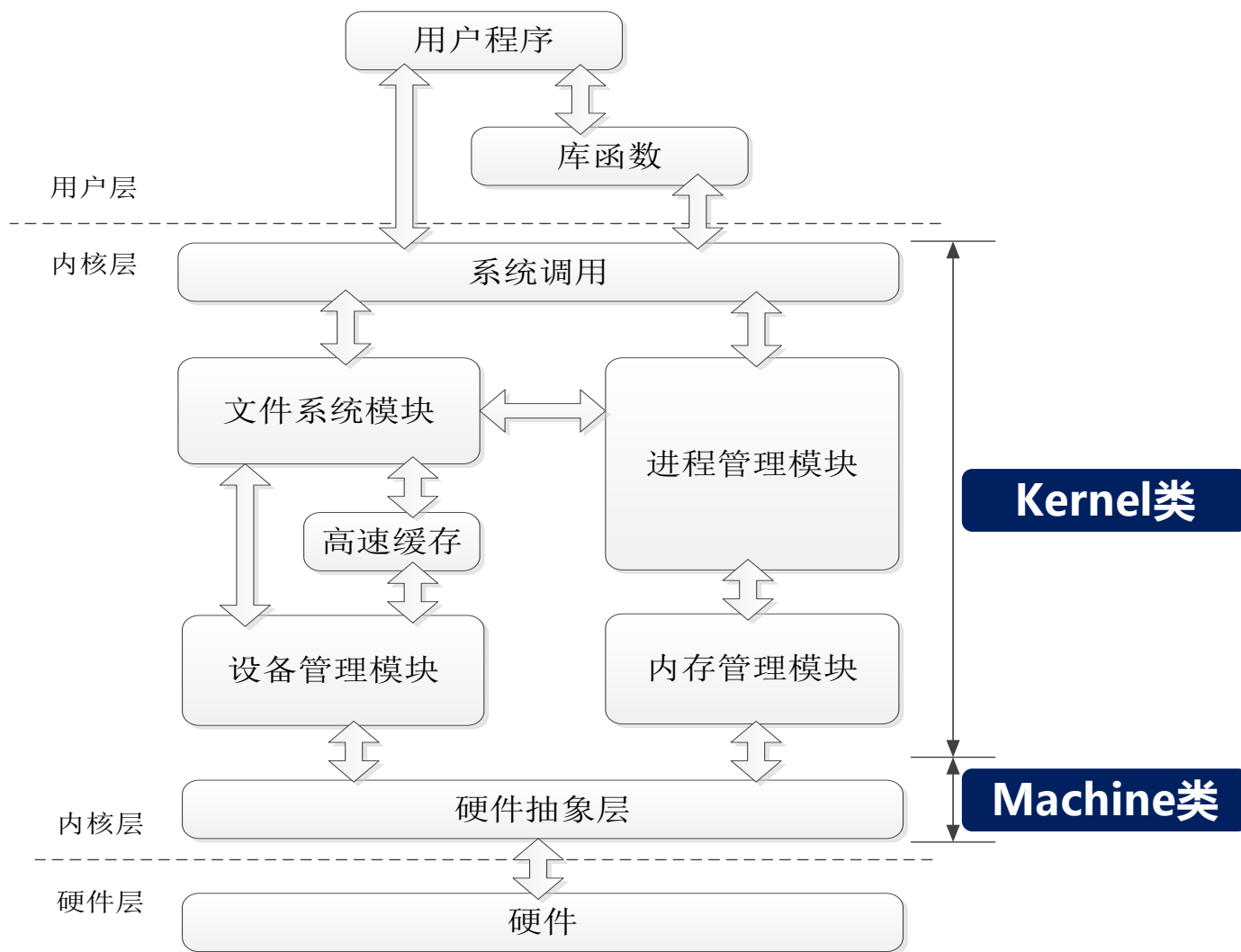


UNIX系统的结构





关于UNIX V6++



```
▼ src
  > boot
  > dev
  > elf
  > fs
  > include
  > interrupt
  > kernel
  > libyosstd
  > machine
  > mm
  > pe
  > proc
  > tty
  > yesa
```



关于I386体系结构

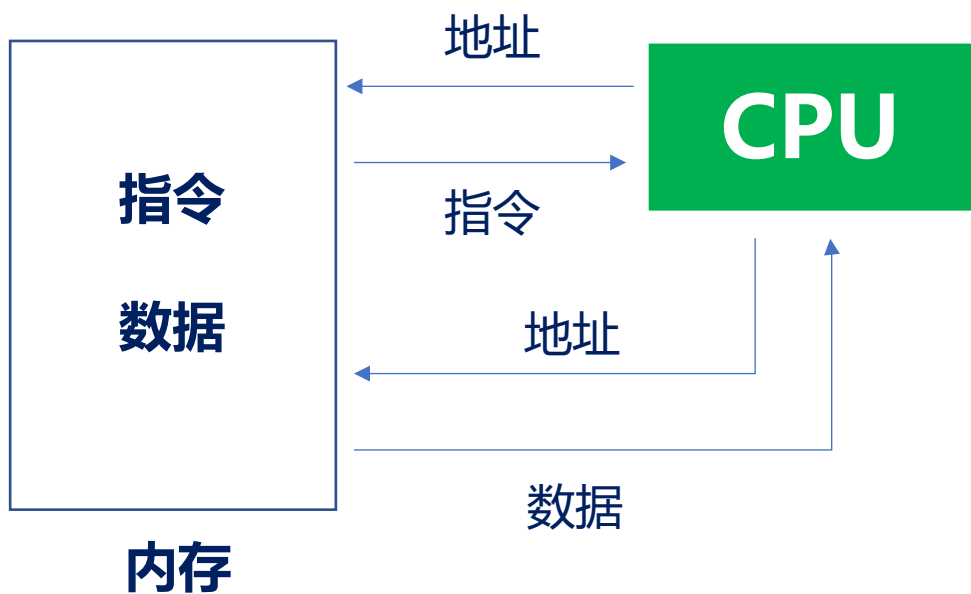


Intel CPU的前世今生



CPU

冯诺依曼体系结构



通用寄存器

变量，中间计算结果，栈指针.....

AX:

BX、CX、DX、SI、DI、SP、BP

专用寄存器

IP: 程序计数器

PSW: 处理机状态字

CS、SS、DS、ES

: 段寄存器



Intel CPU的前世今生



当所有的寄存器都是8位时:

通用寄存器

变量, 中间计算结果, 栈指针.....

AX: 用来存放函数的返回值

BX、CX、DX、SI、DI、SP、BP

循环计数器

字符串操作

栈操作

专用寄存器

IP: 程序计数器

PSW: 处理器状态字



Intel CPU的前世今生



当所有的寄存器都是8位时:

通用寄存器

变量, 中间计算结果, 栈指针.....

AX: 用来存放函数的返回值

BX、CX、DX、SI、DI、SP、BP

循环计数器

字符串操作

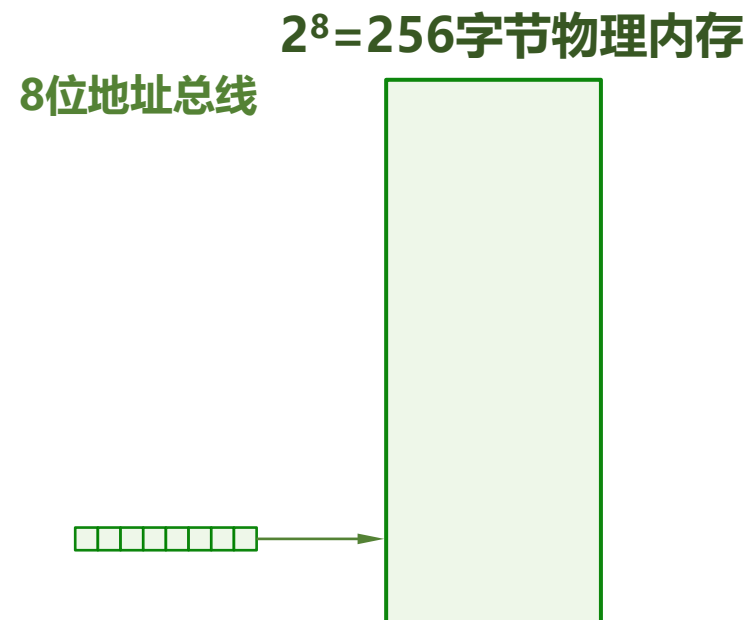
栈操作

专用寄存器

IP: 程序计数器

PSW: 处理器状态字

用寄存器中的地址直接访问物理内存





Intel CPU的前世今生：从实模式到保护模式



8086所有的寄存器都是16位的

通用寄存器

变量，中间计算结果，栈指针.....

AX: 用来存放函数的返回值

BX、CX、DX、SI、DI、SP、BP

循环计数器

字符串操作

栈操作

专用寄存器

IP: 程序计数器

PSW: 处理器状态字

CS、SS、DS、ES

: 段寄存器



Intel CPU的前世今生：从实模式到保护模式



8086所有的寄存器都是16位的

通用寄存器

变量，中间计算结果，栈指针.....

AX: 用来存放函数的返回值

BX、CX、DX、SI、DI、SP、BP

循环计数器

字符串操作

栈操作

专用寄存器

IP: 程序计数器

PSW: 处理器状态字

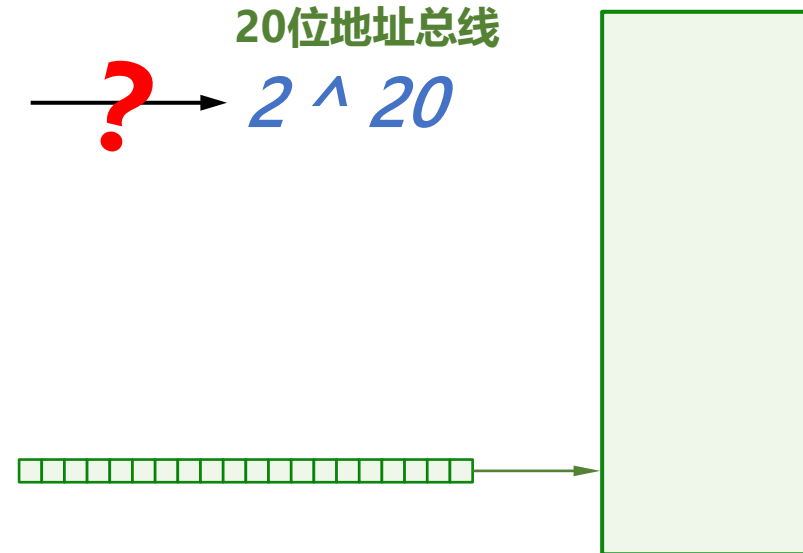
CS、SS、DS、ES

: 段寄存器

$2^{20} = 1\text{M}$ 字节物理内存

$2^{16} \xrightarrow{?} 2^{20}$

20位地址总线





Intel CPU的前世今生：从实模式到保护模式



8086所有的寄存器都是16位的

通用寄存器

变量，中间计算结果，栈指针.....

AX: 用来存放函数的返回值

BX、CX、DX、SI、DI、SP、BP

循环计数器

字符串操作

栈操作

专用寄存器

IP: 程序计数器

PSW: 处理器状态字

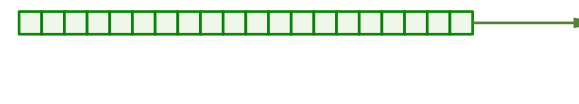
CS、SS、DS、ES

: 段寄存器

$2^{20} = 1\text{M}$ 字节物理内存

$2^{16} \xrightarrow{?} 2^{20}$ 20位地址总线

16位偏移地址寄存器





Intel CPU的前世今生：从实模式到保护模式



8086所有的寄存器都是16位的

通用寄存器

变量，中间计算结果，栈指针.....

AX: 用来存放函数的返回值

BX、CX、DX、SI、DI、SP、BP

循环计数器

字符串操作

栈操作

专用寄存器

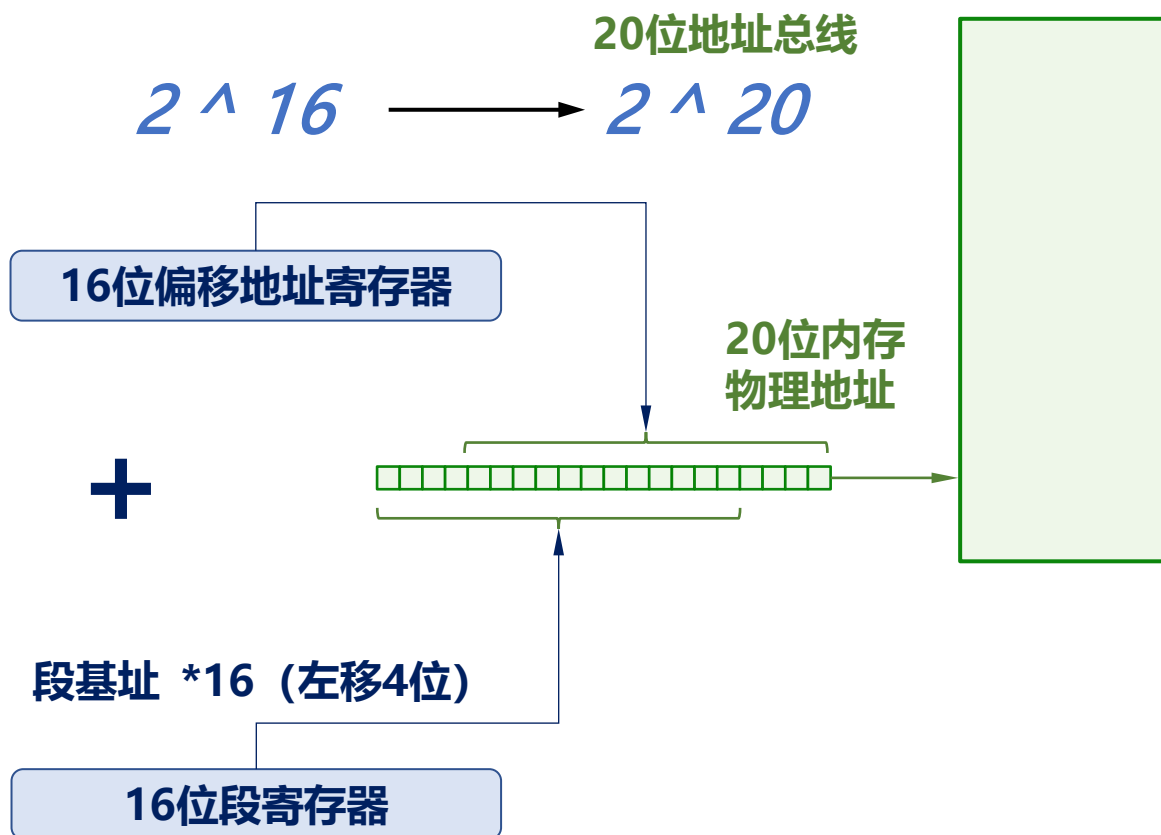
IP: 程序计数器

PSW: 处理器状态字

CS、SS、DS、ES

: 段寄存器

$2^{20} = 1\text{M}$ 字节物理内存



用 “ 16位段寄存器 : 16位偏移地址寄存器 ” 表示程序的逻辑地址

实模式

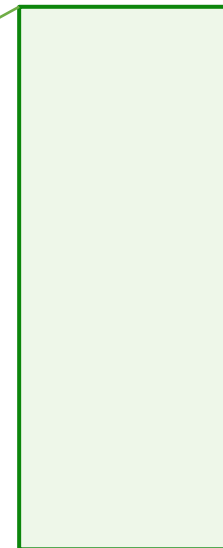


内核的启动过程



$2^{20} = 1\text{M}$ 字节物理内存

起始地址	大小	用途
0x00000	1KB	BIOS的中断向量表
0x00400	256B	BIOS数据区
0x00500	29KB+768B	可用区域
0x07C00	512B	引导扇区数据，被BIOS加载到此处
0x07E00	607KB+512B	可用区域
0x9FC00	1KB	扩展BIOS数据区
0xA0000	64KB	彩色显示适配器缓存
0xB0000	32KB	黑白显示适配器缓存
0xB8000	32KB	文本模式显示适配器缓存
0xC0000	32KB	显示适配器BIOS
0xC8000	160KB	映射硬件适配器ROM或内存映射式的IO
0xF0000	64KB	BIOS程序，入口地址在0xFFFF0





Intel CPU的前世今生：从实模式到保护模式



到了32位寄存器，依然保持这样的设计

通用寄存器

变量，中间计算结果，栈指针.....

EAX: 用来存放函数的返回值

EBX、ECX、EDX、ESI、EDI、ESP、EBP

循环计数器

字符串操作

栈操作

专用寄存器

EIP: 程序计数器

PSW: 处理器状态字

CS、SS、DS、ES、FS、GS、SS: 段寄存器

2^{32}

32位偏移地址寄存器

?

16位段寄存器

用 “ 16位段寄存器 : 16位偏移地址寄存器 ” 表示程序的逻辑地址



Intel CPU的前世今生：从实模式到保护模式



到了32位寄存器，依然保持这样的设计

通用寄存器

变量，中间计算结果，栈指针.....

EAX: 用来存放函数的返回值

EBX、ECX、EDX、ESI、EDI、ESP、EBP

循环计数器

字符串操作

栈操作

专用寄存器

EIP: 程序计数器

PSW: 处理器状态字

CS、SS、DS、ES、FS、GS、SS: 段寄存器

2^{32} $\longrightarrow$ 2^{32} 32位地址总线

32位偏移地址寄存器

?

32位线性地址

16位段寄存器

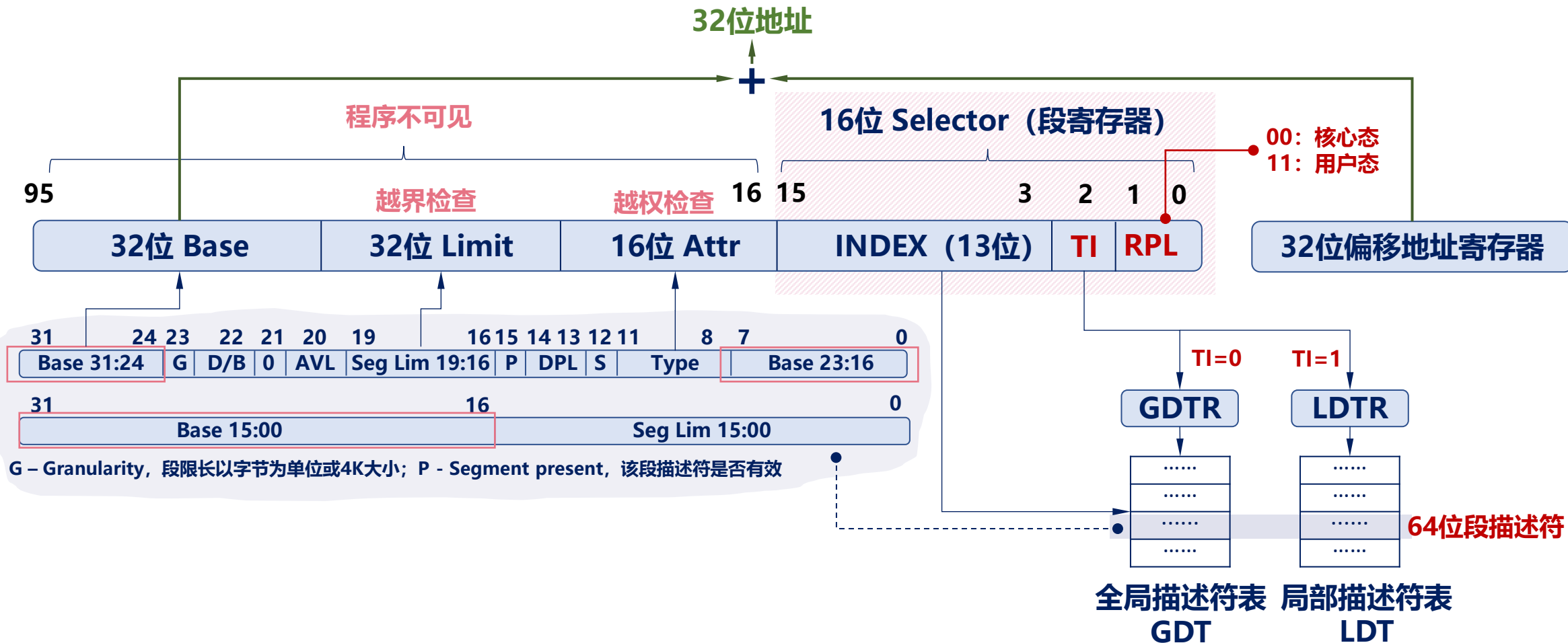
4G虚拟内存

低地址空间的
1M被完整保留

用 “ 16位段寄存器 : 32位偏移地址寄存器 ” 表示程序的逻辑地址



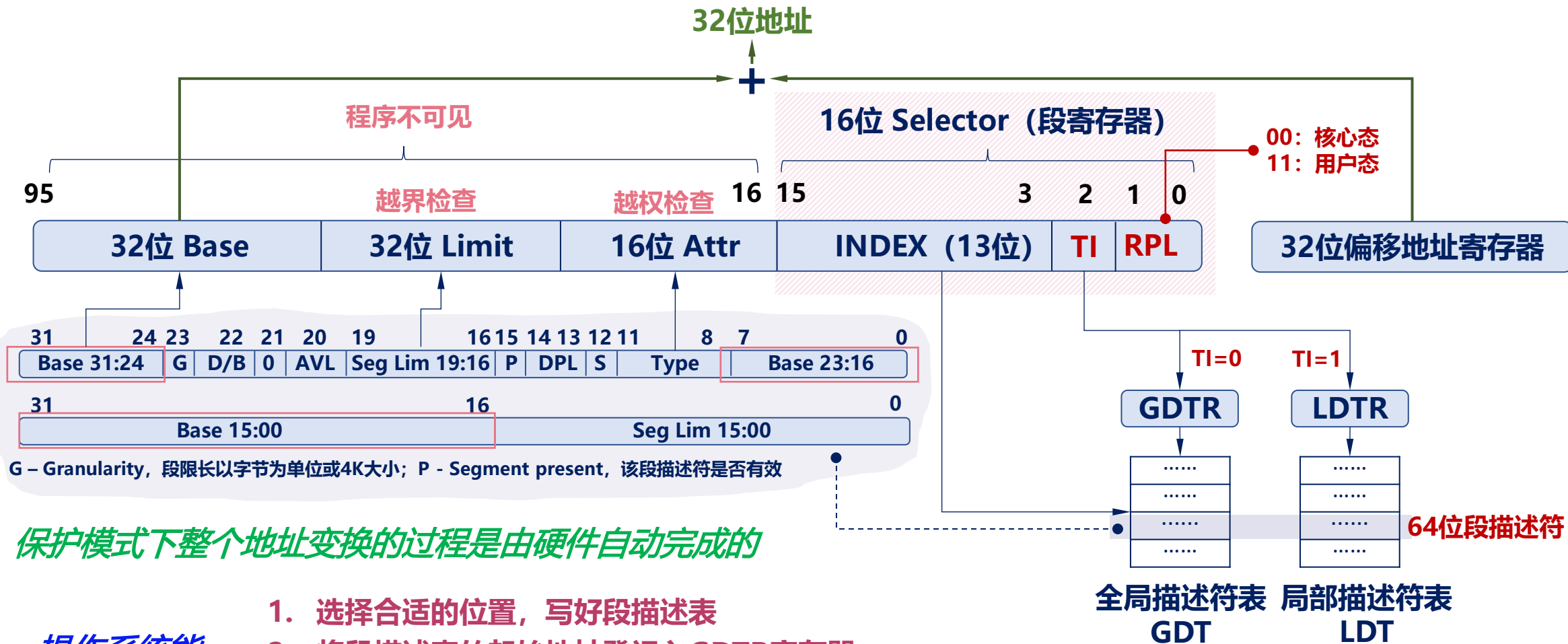
Intel CPU的前世今生：从实模式到保护模式



- 描述符表在内存中, 长度可变
- 最多包含8192 (2^{13})个描述符
- 分别由GDTR和LDTR寄存器保存起始地址



Intel CPU的前世今生：从实模式到保护模式

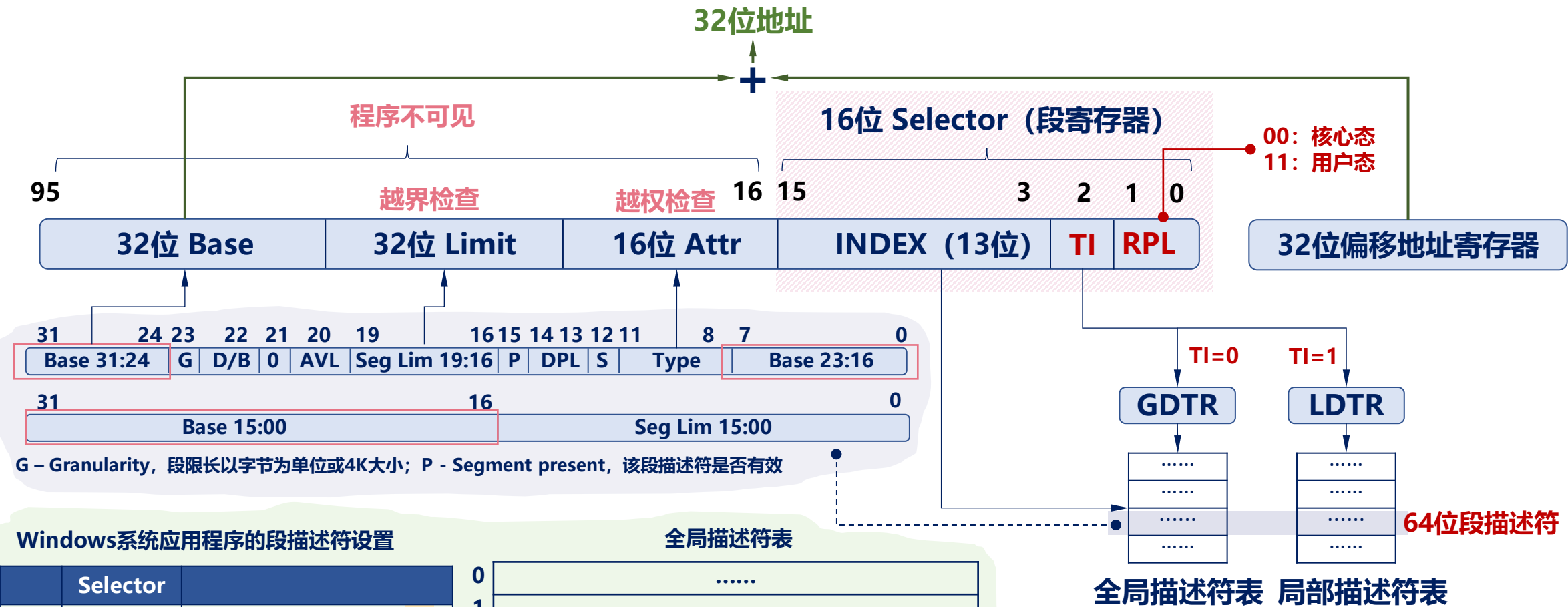


保护模式下整个地址变换的过程是由硬件自动完成的

- 描述符表在内存中，长度可变
- 最多包含8192 (2^{13})个描述符
- 分别由GDTR和LDTR寄存器保存起始地址



Intel CPU的前世今生：从实模式到保护模式



Windows系统应用程序的段描述符设置

	Selector	
CS	001B	0000 0000 0001 1011
ES	0023	0000 0000 0010 0011
SS	0023	0000 0000 0010 0011
DS	0023	0000 0000 0010 0011

全局描述符表

0	
1	
2	
3	Base: 0; Limit: 0xFFFFFFFF; 可读, 可执行
4	Base: 0; Limit: 0xFFFFFFFF; 可读, 可写
5	

- 描述符表在内存中, 长度可变
- 最多包含8192 (2^{13})个描述符
- 分别由GDTR和LDTR寄存器保存起始地址



Intel CPU的前世今生：从实模式到保护模式



到了32位寄存器，依然保持这样的设计

通用寄存器

变量，中间计算结果，栈指针.....

EAX: 用来存放函数的返回值

EBX、ECX、EDX、ESI、EDI、ESP、EBP

循环计数器

字符串操作

栈操作

专用寄存器

EIP: 程序计数器

PSW: 处理机状态字

CS、SS、DS、ES、FS、GS、SS: 段寄存器

$2^{32} \longrightarrow 2^{32}$ 32位地址总线

32位偏移地址寄存器

分段机制

32位线性地址

16位段寄存器

4G虚拟内存

低地址空间的
1M被完整保留

用 “ 16位段寄存器 : 32位偏移地址寄存器 ” 表示程序的逻辑地址

保护模式



Intel CPU的前世今生：从实模式到保护模式



到了32位寄存器，依然保持这样的设计

通用寄存器

变量，中间计算结果，栈指针.....

EAX: 用来存放函数的返回值

EBX、ECX、EDX、ESI、EDI、ESP、EBP

循环计数器

字符串操作

栈操作

专用寄存器

EIP: 程序计数器

PSW: 处理机状态字

CS、SS、DS、ES、FS、GS、SS: 段寄存器

32位偏移地址寄存器

分段机制

16位段寄存器

32位地址总线

$2^{32} \longrightarrow 2^{32}$

32位线性地址

4G虚拟内存

低地址空间的
1M被完整保留

物理内存≠4G
怎么办?

用 “ 16位段寄存器 : 32位偏移地址寄存器 ” 表示程序的逻辑地址

保护模式



Intel CPU的前世今生：从实模式到保护模式



到了32位寄存器，依然保持这样的设计

通用寄存器

变量，中间计算结果，栈指针.....

EAX: 用来存放函数的返回值

EBX、ECX、EDX、ESI、EDI、ESP、EBP

循环计数器

字符串操作

栈操作

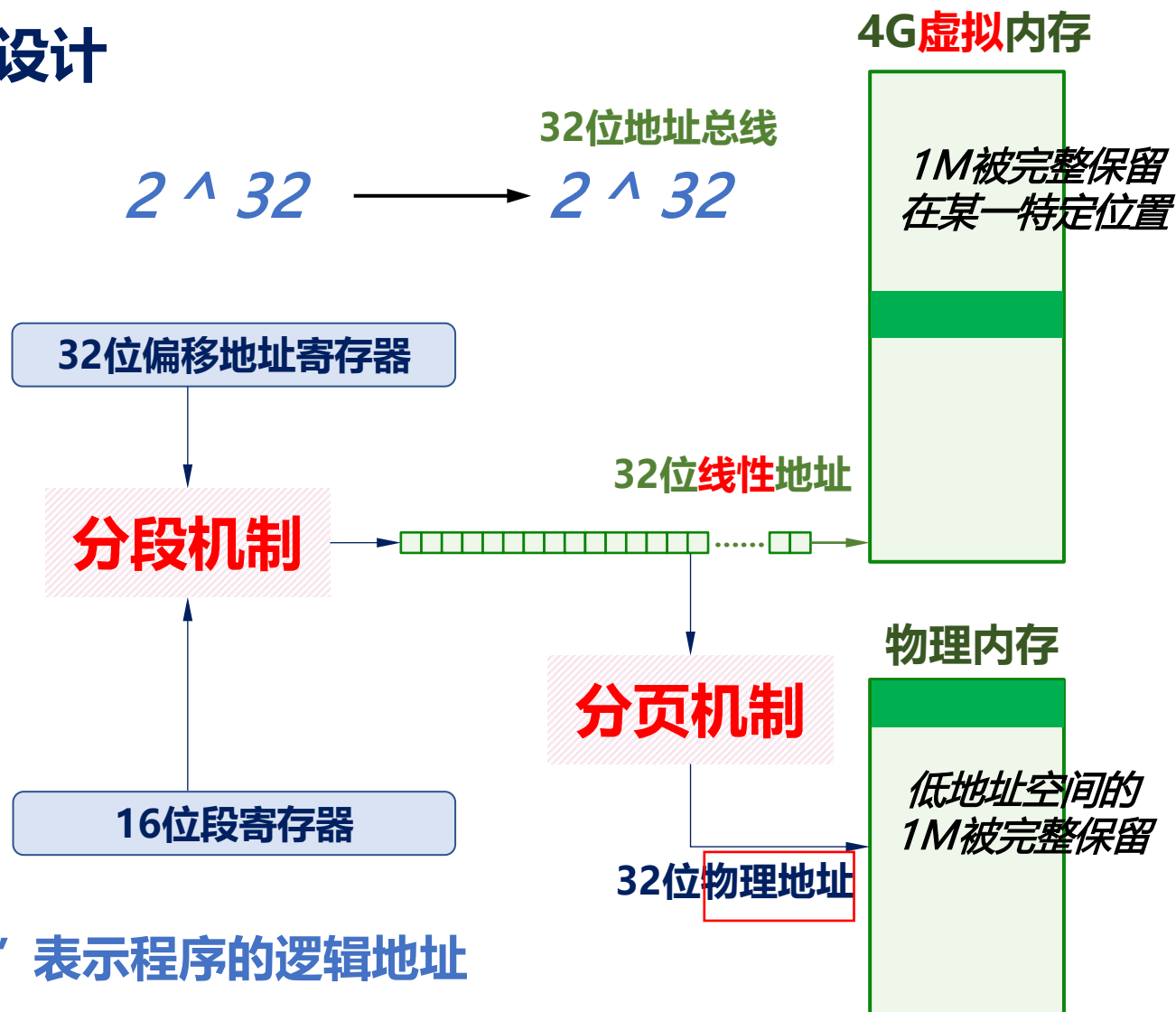
专用寄存器

EIP: 程序计数器

PSW: 处理机状态字

CS、SS、DS、ES、FS、GS、SS: 段寄存器

用 “ 16位段寄存器 : 32位偏移地址寄存器 ” 表示程序的逻辑地址



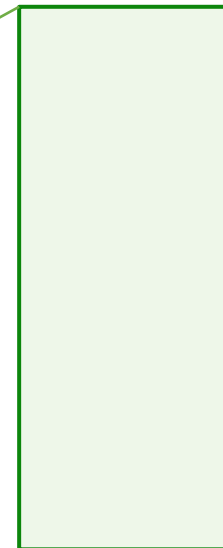


内核的启动过程



$2^{20} = 1\text{M}$ 字节物理内存

起始地址	大小	用途
0x00000	1KB	BIOS的中断向量表
0x00400	256B	BIOS数据区
0x00500	29KB+768B	可用区域
0x07C00	512B	引导扇区数据，被BIOS加载到此处
0x07E00	607KB+512B	可用区域
0x9FC00	1KB	扩展BIOS数据区
0xA0000	64KB	彩色显示适配器缓存
0xB0000	32KB	黑白显示适配器缓存
0xB8000	32KB	文本模式显示适配器缓存
0xC0000	32KB	显示适配器BIOS
0xC8000	160KB	映射硬件适配器ROM或内存映射式的IO
0xF0000	64KB	BIOS程序，入口地址在0xFFFF0





内核的启动过程



$2^{20} = 1\text{M}$ 字节物理内存

起始地址	大小	用途
0x00000	1KB	BIOS的中断向量表
0x00400	256B	BIOS数据区
0x00500	29KB+768B	可用区域
0x07C00	512B	引导扇区数据，被BIOS加载到此处
0x07E00	607KB+512B	可用区域
0x9FC00	1KB	扩展BIOS数据区
0xA0000	64KB	彩色显示适配器缓存
0xB0000	32KB	黑白显示适配器缓存
0xB8000	32KB	文本模式显示适配器缓存
0xC0000	32KB	显示适配器BIOS
0xC8000	160KB	映射硬件适配器ROM或内存映射式的IO
0xF0000	64KB	BIOS程序，入口地址在0xFFFF0

系统上电后/重启后，
执行的第一条指令：

[F000 : FFF0] : 0xFFFF0

ljmp \$0xF000, \$0xE05B



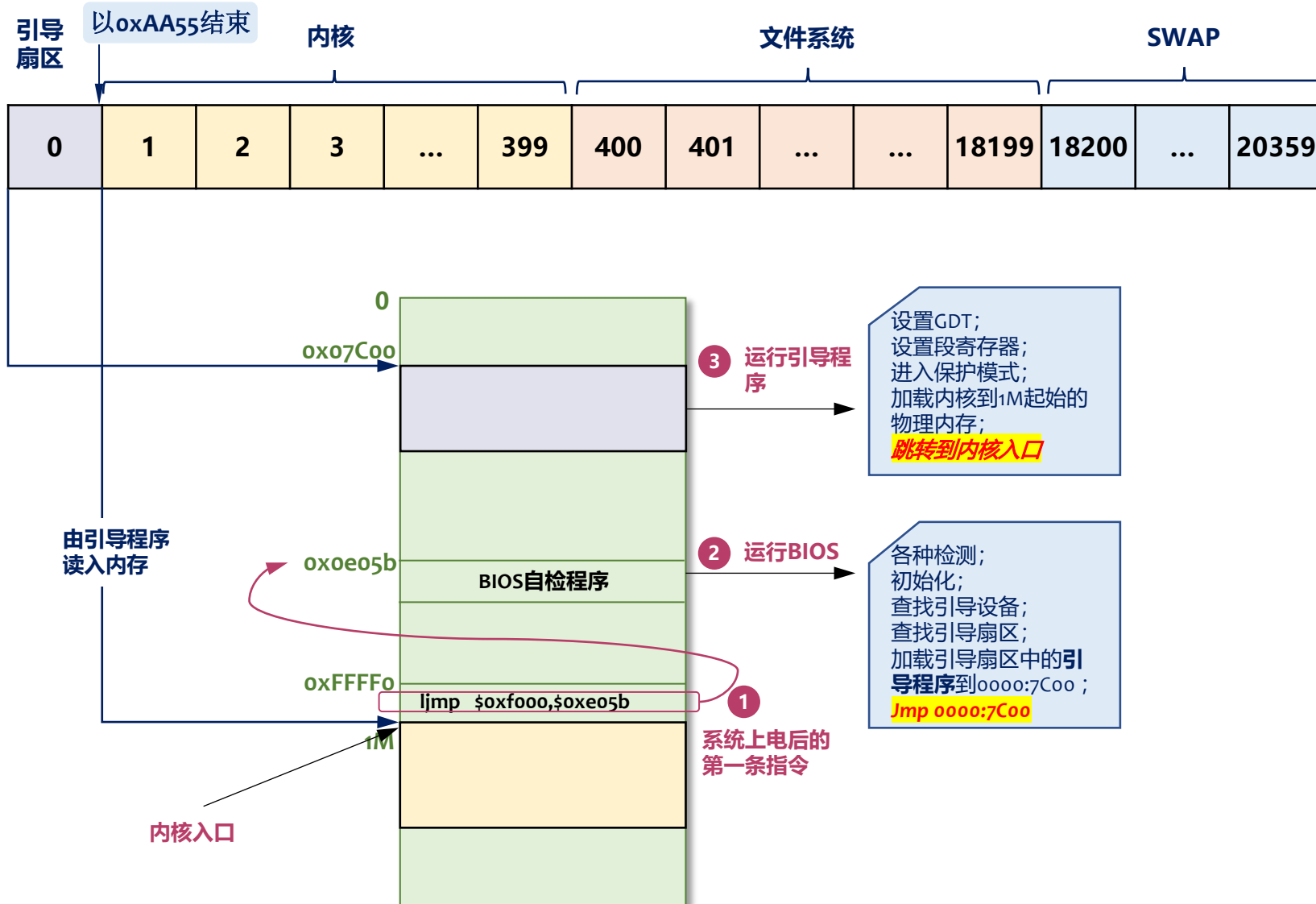
内核的启动过程



1 执行第一条跳转指令

实模式 1M内存

跳转到 `0xF000:0xE05B` 处
(BIOS自检程序的入口)





内核的启动过程



1 执行第一条跳转指令

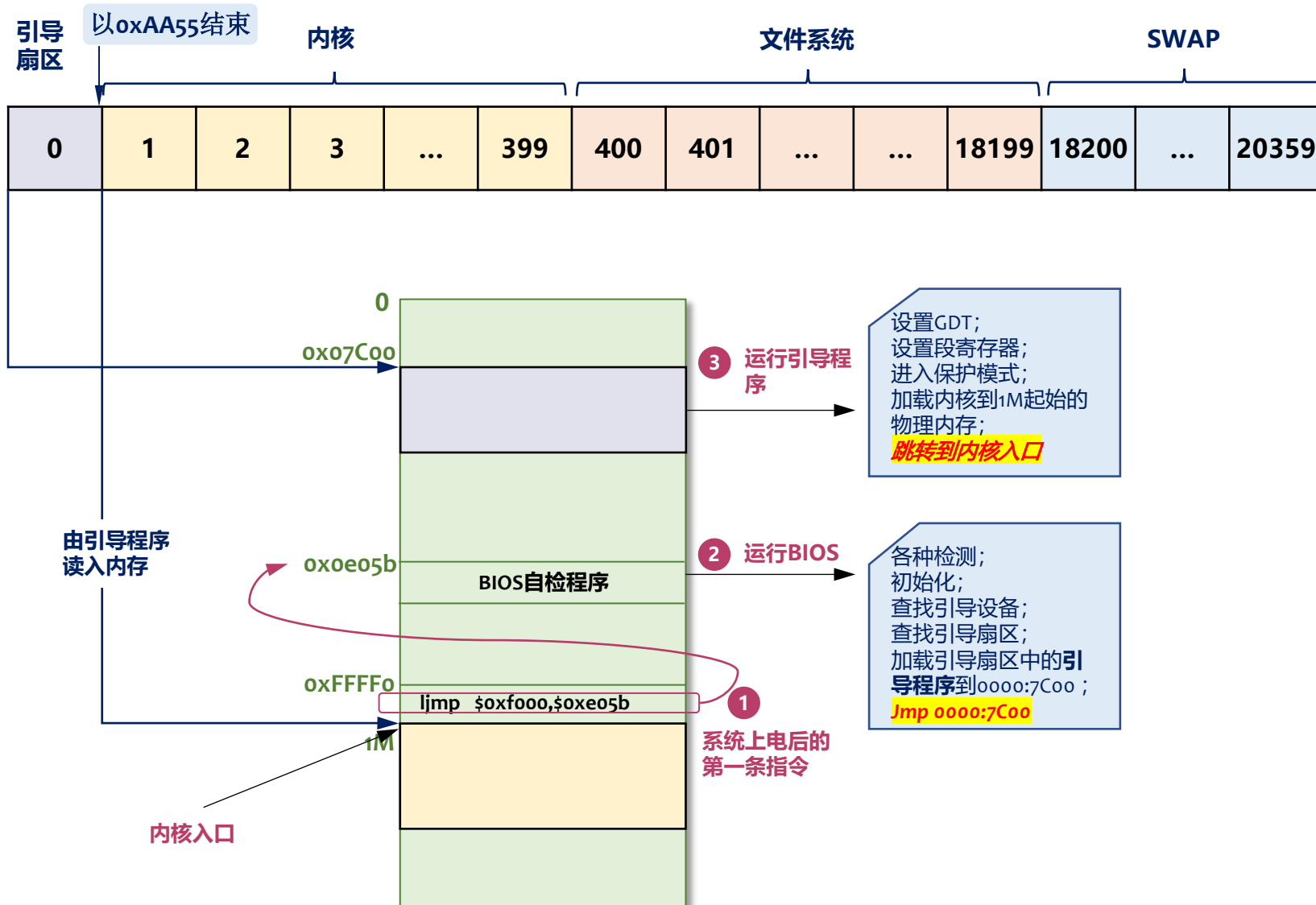
实模式 1M内存

跳转到 `0xF000:0xE05B` 处
(BIOS自检程序的入口)

2 执行BIOS自检程序

实模式 1M内存

对系统硬件检测和初始化后，从引导扇区将引导程序 (BootLoader) 装载到 `0x0000:0x7C00` 处，并跳转到该地址





内核的启动过程



1 执行第一条跳转指令

实模式 1M内存

跳转到 `0xF000:0xE05B` 处
(BIOS自检程序的入口)

2 执行BIOS自检程序

实模式 1M内存

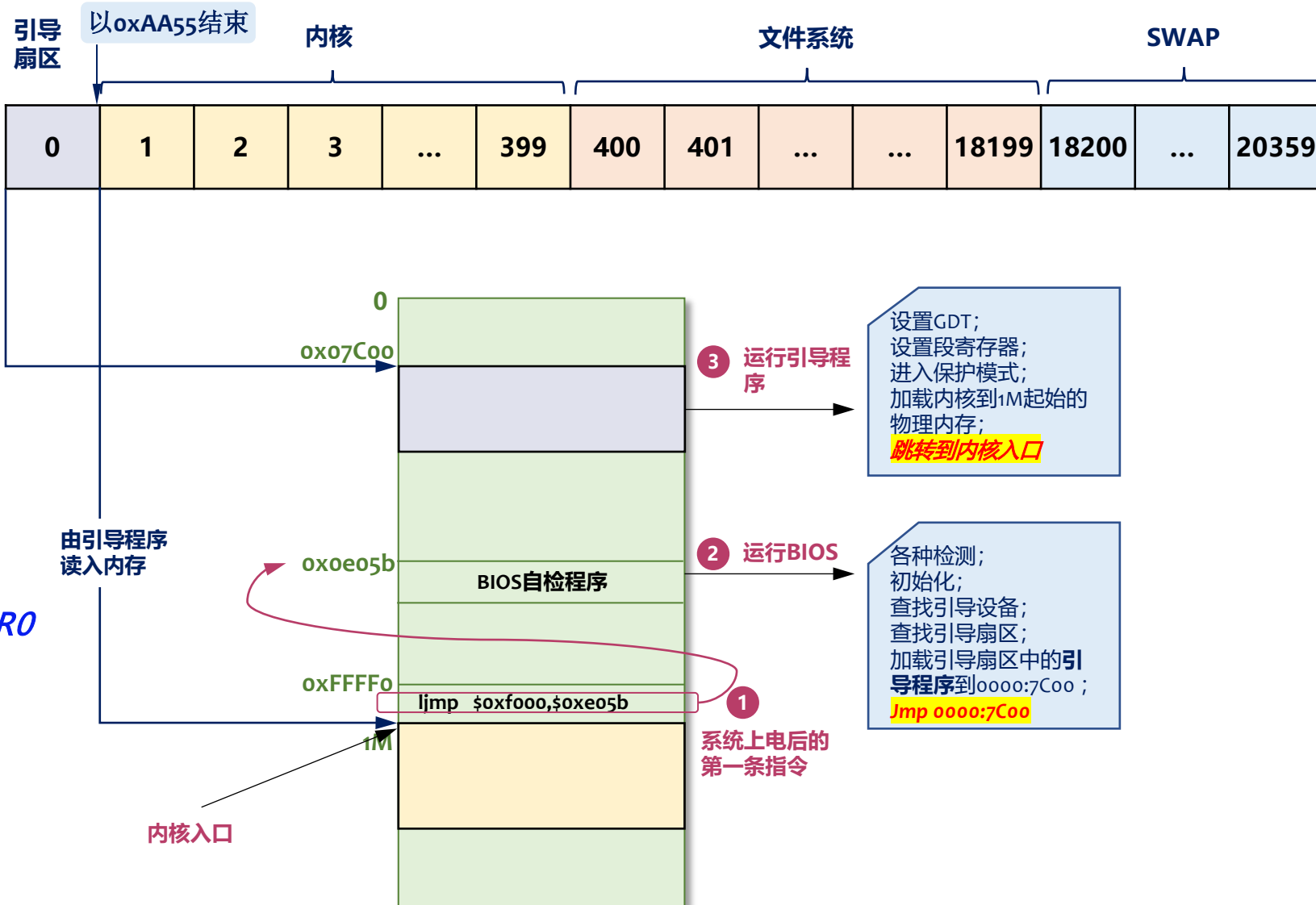
对系统硬件检测和初始化后, 从引导扇区将引导程序 (BootLoader) 装载到 `0x0000:0x7C00` 处, 并跳转到该地址

3 执行BootLoader

启动保护模式 设置 `GDT`, `GDTR`, `CR0`

保护模式 >1M内存

将内核镜像加载到内存1M开始的位置,
做好内存执行的准备工作
并跳转到该位置





本节小结



1 操作系统的主要功能和特征

2 UNIX的基本特征

阅读讲义：1 ~ 18页；23 ~ 28 页



E01：绪论